Memory Efficient Doubly Linked List – Destroy List –Space Complexity Analysis and Algorithm

Program:

```
void destroyList()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        free(curr);
        prev = curr;
        curr = next;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}
…..
       case 9:
            destroyList();
            break;
```

Space Complexity Analysis: *XOR Linked List* (*Destroy List*)

1. *Input Space Complexity*

- ***Parameter − only Definition***:
  - ***Function Parameters***: *None*.
  - ***Analysis***: *No data is passed into the function; it operates solely on the global head.*
  - ***Complexity***: $O(1)$

- ***Full Data Definition***:
  - ***Function Data***: *The entire XOR linked list structure existing in memory.*

  Analysis: *The function must iterate through all n nodes to free them. The input context is the size of the list currently in the heap.*
  - ***Complexity***: $O(n)$

---

2. *Auxiliary Space Complexity*

Explanation: *This measures the temporary memory used by the algorithm during its execution.*

- ***Local Pointer Variables***: *The function uses three local pointers*: *curr*, *prev*, *and next*.
  - ***Analysis***: *These are fixed − size stack variables. Even if the list has a million nodes, we only ever need these three pointers to manage the current deletion and calculate the next address.*

- ***Iterative Process***: *The list is destroyed using a while loop. Since it is not a recursive destruction (which would create $O(n)$ stack frames), it uses constant stack space.*

- ***XOR Operations***: *The next node calculation is done using bitwise math in CPU registers, requiring no extra memory.*

- ***Heap Impact***: *free(curr) removes nodes from the heap. While the* ***Total Memory*** *of the program decreases, the* ***Auxiliary Space*** *(extra memory needed to run the algorithm) does not change.*

Conclusion: *The amount of extra memory used is fixed and does not scale with the number of nodes n.*

- ***Auxiliary Space Complexity***: $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm: *Destroy List* (*XOR Doubly Linked List*)

XORDESTROY(***START***):

1. [***INITIALIZE TRAVERSAL***]
   Set CURRPTR := START
   Set PREVPTR := NULL

2. [***ITERATE THROUGH THE LIST***]
   Repeat while CURRPTR ≠ NULL:

   a. [***DECODE NEXT NODE ADDRESS***]
      Set NEXTTEMPPTR :
      = PREVPTR ⊕ NEXTXORPTR[CURRPTR] (*Calculate the address of the next node before deleting the current one*)

   b. [***SAVE CURRENT ADDRESS AS PREV***]
      Set TEMPPTR := CURRPTR (*Temporarily hold the address of the node to be freed*)

   c. [***FREE CURRENT NODE***]

      FREE(TEMPPTR) (*Release the memory of the current node back to the system*)

   d. [***SHIFT POINTERS FORWARD***]
      Set PREVPTR := TEMPPTR (*Even though the node is freed, its address is still used as the `key` for the next XOR operation*)

      Set CURRPTR := NEXTTEMPPTR

   [*End of Loop*]

3. [***RESET HEAD***]
   Set START := NULL (*Indicate that the list is now empty*)

4. [***PRINT MESSAGE***]
   Write: "*List destroyed.*"

5. ***EXIT***